

Разработка высоконагруженной Event-driven микро сервисной платформы Sport Point для городских спортивных ивентов

Шабанов Максим

Тема проекта

Разработка высоконагруженной Event-driven микро сервисной платформы для создания, поиска и участия в городских спортивных ивентах с интерактивной картой, системой профилей и асинхронным взаимодействием между сервисами.



Суть проекта

Sport Point — это платформа, которая объединяет активных людей города.
Основные возможности:

- Карта ивентов — пользователи видят все мероприятия на интерактивной карте города
 - Создание ивентов — любой пользователь может организовать событие (пробежка, турнир, тренировка и т.д.)
 - Подписка и участие — можно записаться на интересующее ивент и увидеть количество других участников и количество свободных мест
 - Профиль пользователя — каждый может рассказать о себе, своих интересах и спортивной активности
- 

Стек технологий

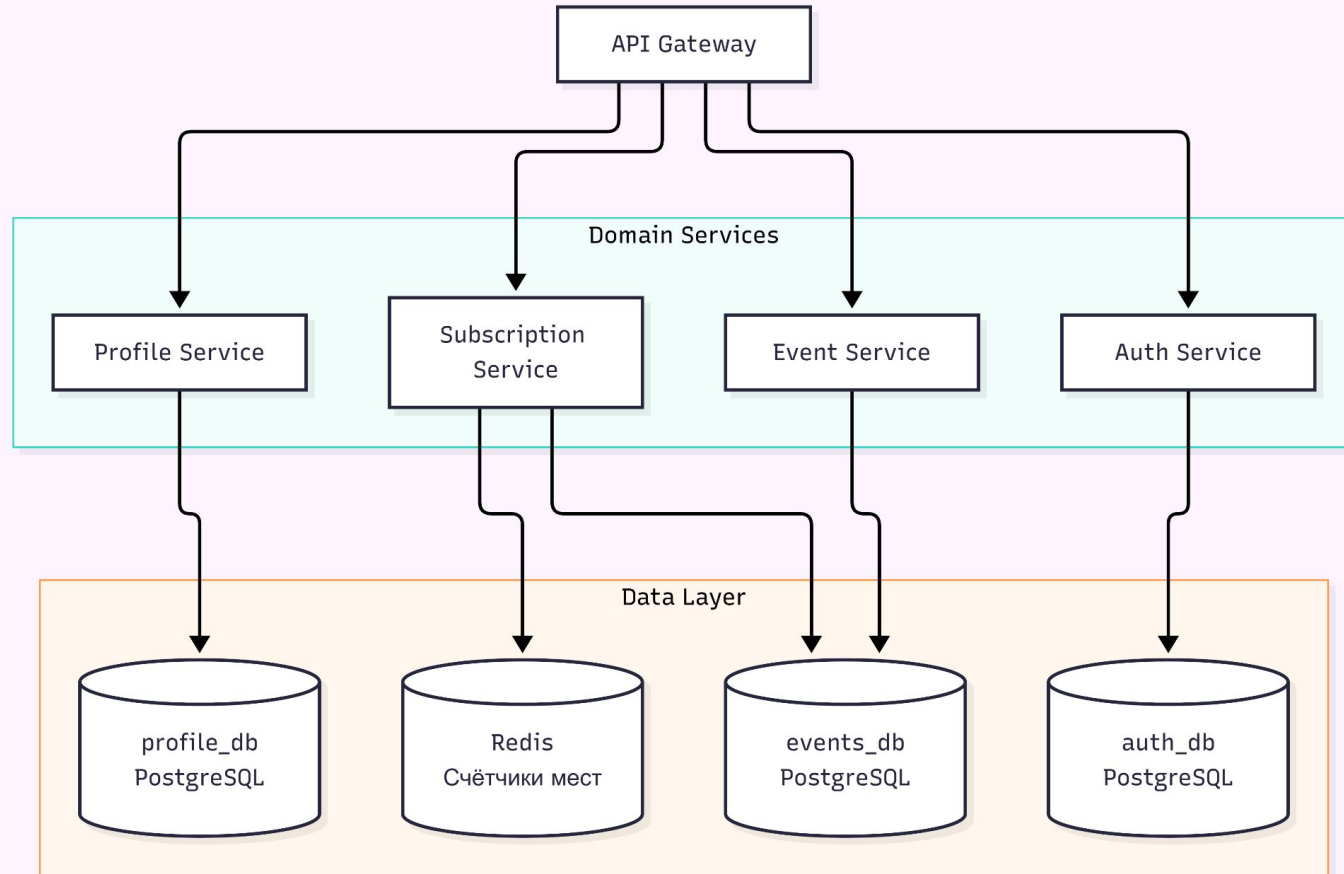
Категория	Технологии
Языки и фреймворки	Java, Spring Boot, Spring WebFlux, TypeScript, React
Работа с данными	Hibernate, Spring Data JPA, Spring Data Redis, Spring Kafka
Базы данных и кэш	PostgreSQL, Redis
Брокер сообщений	Apache Kafka
Инфраструктура	VPS, Docker, Nginx
Безопасность	Spring Security, JWT
Системы сборки	Maven, Vite
UI библиотеки	Ant Design, Leaflet

Структура проекта

Проект построен по принципу Database-per-Service. Каждый сервис владеет своей предметной областью и своими данными.



Sport Point Platform



Детальная таблица сервисов

Микросервис	Ответственность	База данных	Эндпоинты
Gateway	Маршрутизация, JWT-валидация, обогащение заголовков	-	-
Auth Service	Регистрация, аутентификация, выпуск JWT, менеджмент пользователей	auth_db (PostgreSQL)	/register, /login, /users/**
Profile Service	Менеджмент профилей	profile_db (PostgreSQL)	/profiles/**
Event Service	Менеджмент ивентов	events_db (PostgreSQL)	/events/**
Subscription Service	Подписки на ивенты, менеджмент количества мест в ивентах	events_db (PostgreSQL) + Redis	/subscriptions/**

⚡ Redis (только Subscription Service)

Ключ: event:
{eventId}:subs
Тип: Integer
Операции: DECR
/ INCR
(атомарные)

📅 events_db

events

- id, title
- date, lat/lng
- organizer_id
- status

subscriptions

- id, user_id
- event_id
- status

👤 profile_db

profiles

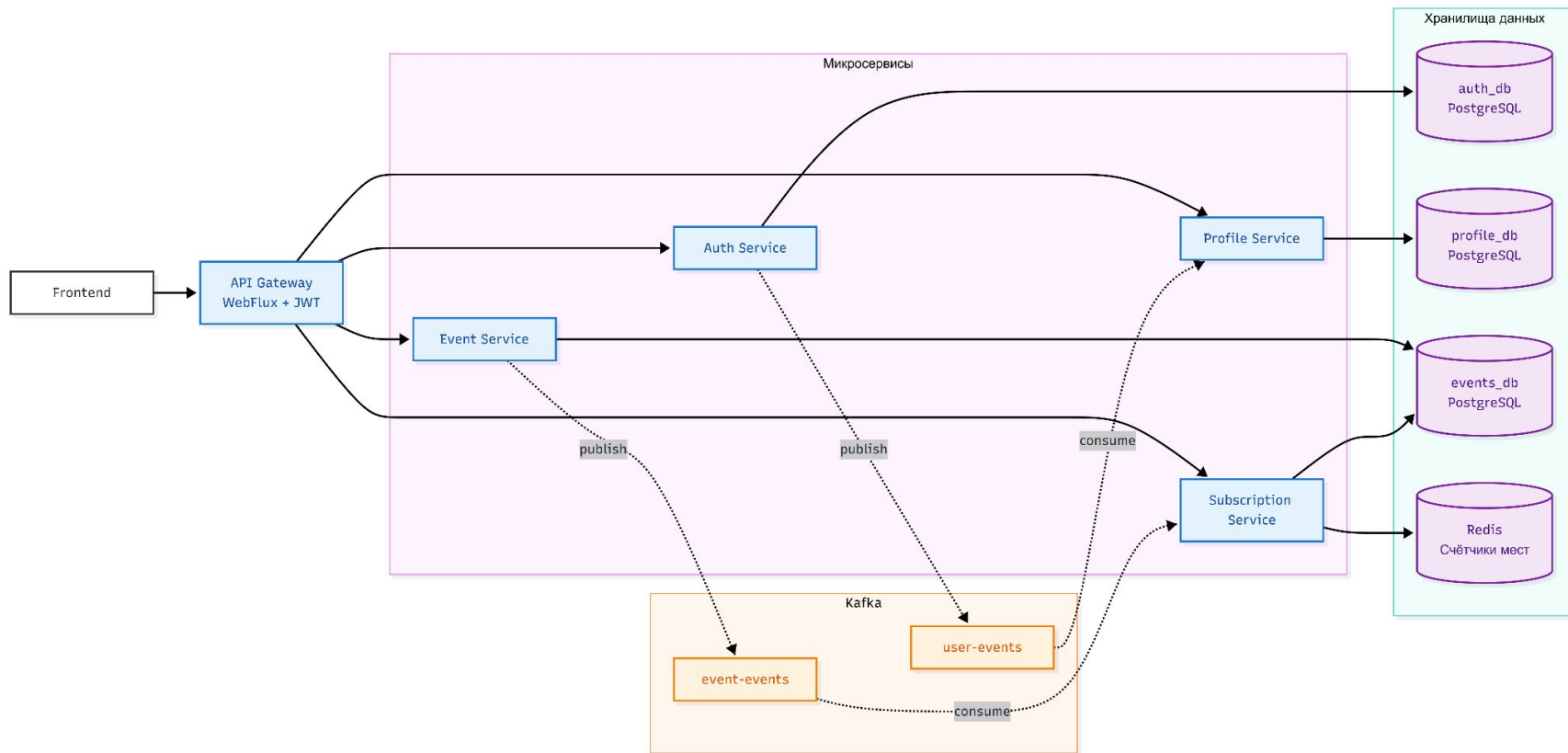
- id
- user_id
- bio

🔒 auth_db

users

- id
- username
- password
- role
- created_at

Схема базы данных



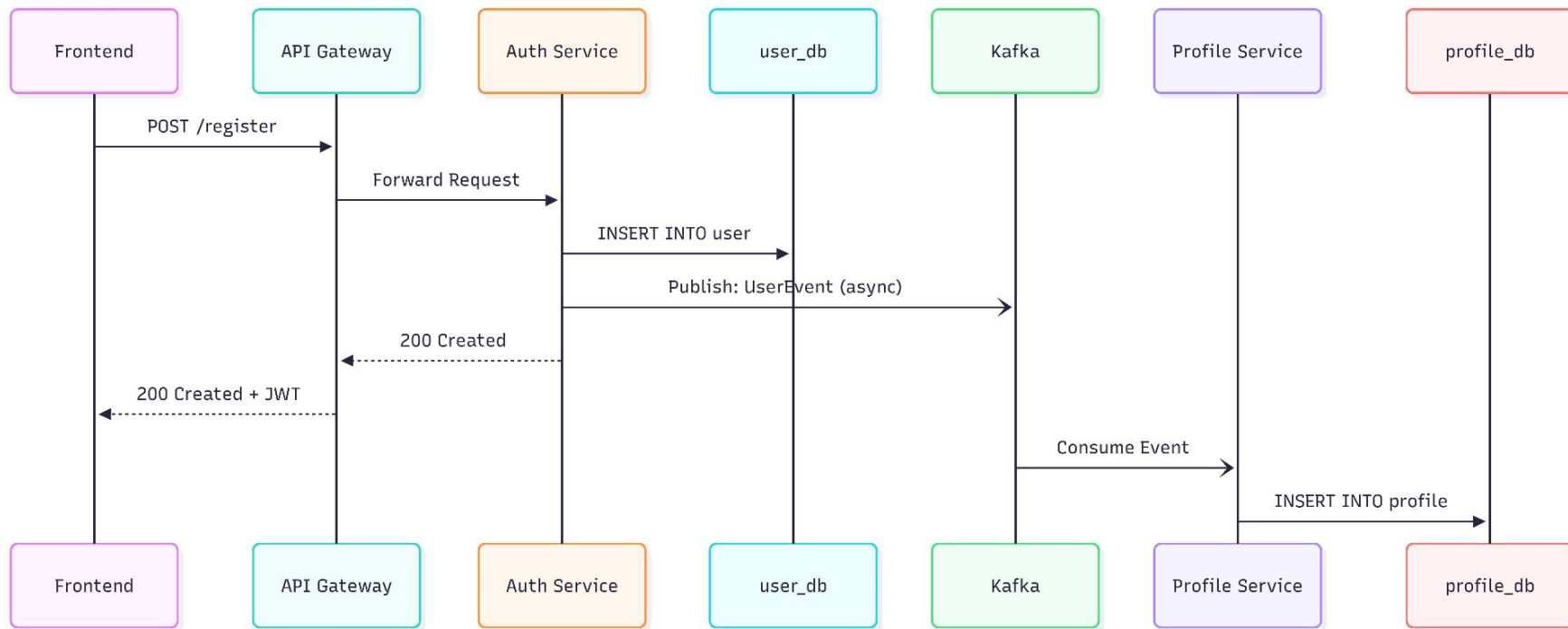
Общая схема взаимодействия

Архитектурный принцип

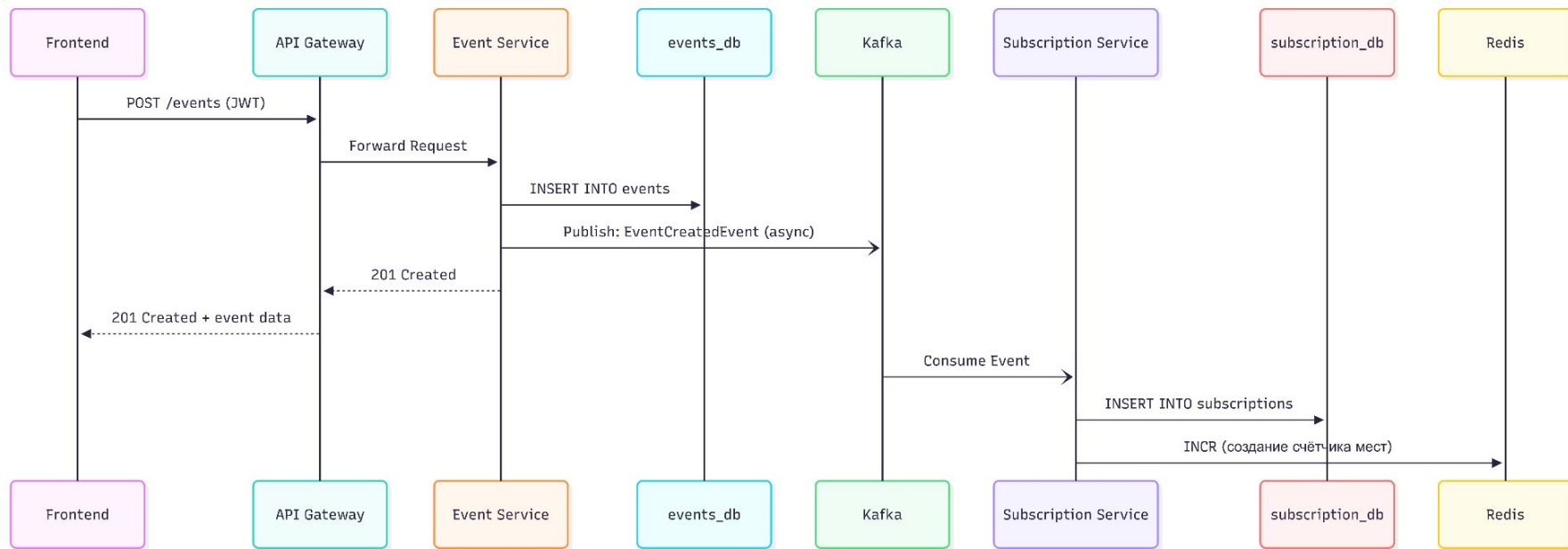
- Синхронный вход — клиент всегда получает мгновенный ответ от сервиса, которому адресован запрос
- Асинхронный "хвост" — каскадные операции (создание профиля, создание инвента) выполняются через Kafka
- Атомарность критичных операций — Redis гарантирует корректную работу с местами даже под нагрузкой
- Слабая связанность — сервисы не знают друг о друге, общаются только через события
- Отказоустойчивость — падение одного сервиса не валит всю систему
- Масштабируемость — каждый сервис можно масштабировать независимо



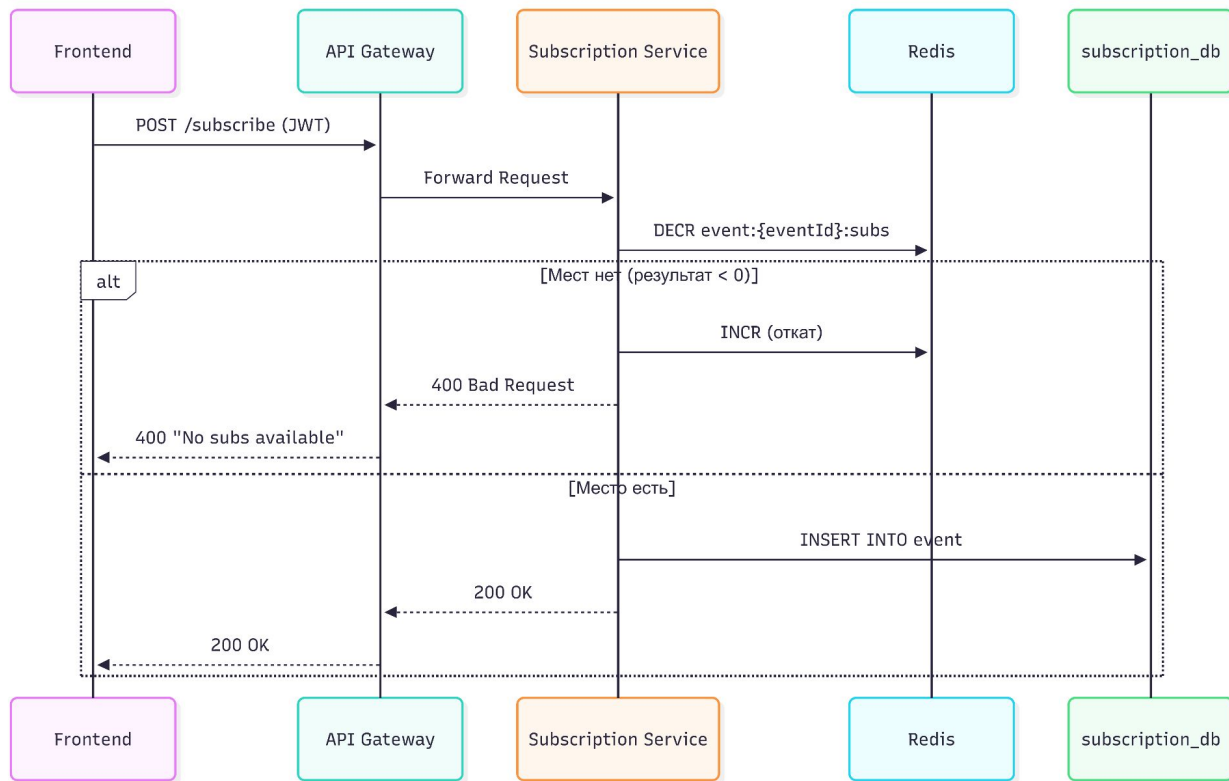
Бизнес-сценарии и Основные use-case



Сценарий 1: Регистрация пользователя и создание профиля



Сценарий 2: Создание ивента на карте



Сценарий 3: Подписка пользователя на ивент